How to Run This Project — Step-by-Step

Nawaloka Hospital AI Assistant — a complete local run guide, from cloning the repo to chatting with the assistant in your browser.

Follow the steps in order. Each step says exactly what to type and what you should see. By the end you'll have the full stack running locally: a web chat UI, a FastAPI backend, RAG (Qdrant), CRM + memory (Supabase), and an optional voice pipeline.

What You'll End Up With

Piece	URL / Where	What it is
Web chat UI	http://localhost:8080	The assistant you talk to
API + docs	http://localhost:8000/docs	The FastAPI backend
Redis	localhost:6379	Short-term cache (runs in Docker)
Supabase	cloud	CRM data (doctors, patients, bookings) + memory
Qdrant	cloud	RAG knowledge base (hospital info)

---

0. Prerequisites

Install these **before** you start. Check each one is working with the command shown.

Tool	Why you need it	Check it works
Docker Desktop	Runs the app containers	`docker --version`
Git	Clone the repo	`git --version`
Python 3.10+	Runs the data-seeding scripts	`python3 --version`
Make	Shortcut commands (pre-installed on macOS/Linux)	`make --version`

> **Important:** Make sure **Docker Desktop is actually running** (you see the whale icon in your menu bar / system tray) before any `docker` command. If `docker ps` errors with "Cannot connect to the Docker daemon", Docker Desktop isn't up yet.

> **Windows users:** run all commands inside **WSL2** (Ubuntu) or **Git Bash**, not PowerShell — the `make` targets assume a Unix shell.

---

1. Clone the Repository

```
git clone <repo-url>
cd "E2E Deployment"
```

Replace `<repo-url>` with the URL you were given. You should now be inside the project folder (the one that contains `Makefile`, `docker-compose.yml`, and `.env.example`).

---

2. Create Your `.env` File

The app reads all its secrets from a `.env` file. We ship a template called `.env.example` with every key spelled out. Copy it:

```
cp .env.example .env
```

Now open `.env` in any editor and fill in your **own** keys. Here's where each one comes from:

Key(s)	Service	Get it from
`QDRANT_API_KEY`, `QDRANT_URL`	Qdrant Cloud (RAG)	https://cloud.qdrant.io
`SUPABASE_DB_URL`, `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`	Supabase (database)	https://supabase.com → your project → Settings
`OPENAI_API_KEY`	OpenAI (embeddings)	https://platform.openai.com/api-keys
`GROQ_API_KEY`	Groq (fast LLM)	https://console.groq.com/keys
`OPENROUTER_API_KEY`	OpenRouter (LLM fallback)	https://openrouter.ai/keys
`TAVILY_API_KEY`	Tavily (web search)	https://app.tavily.com
`LANGFUSE_*`	Langfuse (tracing) — *optional*	https://cloud.langfuse.com
`LIVEKIT_*`, `DEEPGRAM_API_KEY`, `ELEVEN_API_KEY`	Voice — *only if you run voice*	See Step 7

> **Two things to know:**
>
> 1. For `SUPABASE_DB_URL`, use the **transaction pooler** on port `6543` (not `5432`) — the example file already shows the right format.
> 2. `REDIS_URL` should stay as `redis://redis:6379/0` when running with Docker (that's the container name). The example file already has this.

> ⚠️ **Never commit your `.env` file.** It holds your real secrets and is already in `.gitignore`. Only `.env.example` (placeholders) belongs in git.

---

3. Set Up a Python Environment

The data-seeding scripts (Step 4) run on your machine, so you need Python dependencies installed. Create an isolated environment so you don't pollute your system Python:

```
python3 -m venv .venv
source .venv/bin/activate
make install
```

- `python3 -m venv .venv` — creates a `.venv/` folder (the scripts expect this exact name).
- `source .venv/bin/activate` — activates it. Your prompt now starts with `(.venv)`.
- `make install` — installs everything from `requirements.txt`.

> **Re-activate after closing your terminal:** every new terminal session, run `source .venv/bin/activate` again before any `make` data command.

---

4. Seed the Data

Now fill your cloud databases with hospital data. Run these **four** commands in order (one-time setup):

```
make init-supabase      # 1. Create the database tables (doctors, patients,
bookings, memory)
make seed-crm-large     # 2. Generate 10 doctors + 20 patients + appointment
slots
make seed-procedures    # 3. Load procedural workflows (how to
book/reschedule/cancel)
make ingest-qdrant      # 4. Load the hospital knowledge base into Qdrant
(for RAG)
```

What each one does:

1. **`make init-supabase`** — builds the database schema in Supabase (tables + pgvector indexes + helper functions). Run this **once**.
2. **`make seed-crm-large`** — uses an LLM to generate realistic doctors, patients, and bookings (~30 seconds, costs ~$0.01). *No API budget? Use `make seed-crm-no-llm` instead — it's instant and free.*
3. **`make seed-procedures`** — loads the step-by-step workflows the assistant follows for CRM actions.
4. **`make ingest-qdrant`** — chunks the markdown files in `data/knowledge_base/` and uploads them to Qdrant so the assistant can answer factual questions.

Verify the data landed:

```
make query-crm      # Should print non-zero counts for doctors, patients,
bookings
make qdrant-info    # Should show your collection with ~130+ points
```

5. Run the App

With data seeded and `.env` filled, bring up the stack in Docker:

```
make demo
```

This runs `docker compose up --build -d` — it builds the images and starts **redis + api + web** in the background. The first build takes a few minutes; after that it's seconds.

You'll see:

```
✅  Stack is up. First boot takes ~60s for lifespan warmup.
    Web UI →  http://localhost:8080
    API    →  http://localhost:8000  (docs at /docs)
```

Wait ~60 seconds for the API to finish warming up, then open **http://localhost:8080** in your browser and start chatting.

Try these to see each subsystem light up:

- "*What are the opening hours?*" → **RAG** (knowledge base)
- "*Book me an appointment with a cardiologist*" → **CRM** (database)
- "*What's the latest news on diabetes treatment?*" → **Web search**

---

6. Check Everything Is Healthy

```
docker compose ps      # All services should say "running" / "healthy"
make demo-logs         # Tail the API logs (Ctrl+C to stop watching)
```

Quick API health check:

```
curl http://localhost:8000/healthz
```

If the web UI loads but chat replies are slow or error on the first message, give it another 30–60 seconds — the API warms up its model clients on first boot.

---

7. (Optional) Run the Voice Assistant

The voice pipeline is a **side-car** — the chat app works fine without it. To enable voice you need three more keys in `.env`:

Key	Service	Get it from
`LIVEKIT_URL`, `LIVEKIT_API_KEY`, `LIVEKIT_API_SECRET`	LiveKit Cloud (audio rooms)	https://cloud.livekit.io
`DEEPGRAM_API_KEY`	Deepgram (speech-to-text)	https://console.deepgram.com
`ELEVEN_API_KEY`	ElevenLabs (text-to-speech)	https://elevenlabs.io

First validate your voice config:

```
make voice-test      # Prints STT/TTS settings + confirms env vars are
present
```

Then bring up the stack **with** the voice worker:

```
make demo-voice
```

To talk to it, open the LiveKit Agents Playground and connect to your project:

> https://agents-playground.livekit.io

Watch voice logs with `make voice-logs`. Stop the voice stack with `make demo-voice-down`.

---

8. Stopping & Cleaning Up

```
make demo-down              # Stop the app (keeps your data)
make demo-voice-down        # Stop the voice stack
docker compose down -v      # Stop AND wipe Docker volumes (caches) — rarely
needed
```

Your **data is safe** in Supabase and Qdrant (the cloud) — stopping the containers doesn't delete it. You only need to re-run Step 4 if you want to reset or re-seed.

Command Cheat-Sheet

Run `make help` any time to see every available command. The ones you'll actually use:

Command	What it does
`make install`	Install Python dependencies
`make init-supabase`	Create database tables (run once)
`make seed-crm-large`	Seed doctors/patients/bookings (LLM)
`make seed-crm-no-llm`	Seed the same, but free + instant
`make seed-procedures`	Load CRM action workflows
`make ingest-qdrant`	Load knowledge base into Qdrant
`make ingest-qdrant-recreate`	Wipe + re-load the knowledge base
`make query-crm`	Print database row counts
`make qdrant-info`	Print Qdrant collection stats
`make status`	Show what data sources are configured
`make demo`	**Run the app** (api + web)
`make demo-logs`	Tail the API logs
`make demo-down`	Stop the app
`make demo-voice`	Run the app **+ voice worker**
`make voice-logs`	Tail the voice worker logs
`make demo-voice-down`	Stop the voice stack

Troubleshooting

Symptom	Likely cause	Fix
`Cannot connect to the Docker daemon`	Docker Desktop isn't running	Start Docker Desktop, wait for the whale icon, retry
❌ `No .env found` when running `make demo`	You skipped Step 2	`cp .env.example .env` and fill in your keys
`make init-supabase` fails to connect	Wrong `SUPABASE_DB_URL`	Use the **pooler** URL on port **6543**, not 5432
`make` script says `python: command not found`	venv not activated	`source .venv/bin/activate`, then retry
Web UI loads but chat errors on first message	API still warming up	Wait 60s, then retry; check `make demo-logs`
`make seed-crm-large` errors / no LLM budget	Missing/limited LLM key	Use `make seed-crm-no-llm` (free, no API)
Port 8080 or 8000 "already in use"	Another app (or an old run) holds the port	`make demo-down`, then `make demo` again
RAG answers are empty / "I don't know"	Knowledge base not ingested	Run `make ingest-qdrant`, confirm with `make qdrant-info`

The One-Page Version

Already set up once? Here's the whole thing start to finish:

```
# One-time setup
git clone <repo-url> && cd "E2E Deployment"
cp .env.example .env                # then fill in your keys
python3 -m venv .venv && source .venv/bin/activate
make install
make init-supabase
make seed-crm-large
make seed-procedures
make ingest-qdrant

# Run it (every time)
make demo                           # → http://localhost:8080
make demo-down                      # when you're done
```

Nawaloka Hospital AI Assistant — AI Engineer Essentials. Questions? Check `README.md` or ask in the cohort channel.